

```

#include <iostream>
using namespace std;

const int MAX = 100;
int visited[MAX];
int startTime[MAX];
int finishTime[MAX];
int time_counter = 0;

class Stack {
    int arr[MAX];
    int top;
public:
    Stack() { top = -1; }
    bool isEmpty() { return top == -1; }
    bool isFull() { return top == MAX - 1; }
    void push(int val) {
        if (isFull()) { cout << "Stack is full\n"; return; }
        arr[++top] = val;
    }
    int pop() {
        if (isEmpty()) return -1;
        return arr[top--];
    }
    int peek() {
        if (isEmpty()) return -1;
        return arr[top];
    }
};

// Iterative DFS, records start/finish time
void DFS(int adj[MAX][MAX], int n, int SN) {
    int nextChild[MAX] = {0};
    Stack st;

    st.push(SN);
    visited[SN] = 1;
    startTime[SN] = ++time_counter;
    cout << SN + 1 << " ";
    nextChild[SN] = 0;

    while (!st.isEmpty()) {
        int cur = st.peek();
        bool found = false;

        while (nextChild[cur] < n) {
            int next = nextChild[cur];
            nextChild[cur]++;

            if (adj[cur][next] == 1 && visited[next] != 1) {
                st.push(next);
                visited[next] = 1;
                startTime[next] = ++time_counter;
                cout << next + 1 << " ";
                nextChild[next] = 0;
                found = true;
                break;
            }
        }
    }
}

```

```

        if (!found) {
            st.pop();
            finishTime[cur] = ++time_counter;
        }
    }
}

void inputEdgesToAdjMatrix(int adj[MAX][MAX], int n, int e) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            adj[i][j] = 0;

    cout << "Enter edges (source destination):\n";
    for (int i = 0; i < e; i++) {
        int u, v;
        cin >> u >> v;
        if (u >= 1 && u <= n && v >= 1 && v <= n &&
            u != v && adj[u - 1][v - 1] == 0) {
            adj[u - 1][v - 1] = 1;
            // Undirected? also: adj[v-1][u-1] = 1;
        } else {
            cout << "Invalid edge\n";
            i--;
        }
    }
}

int main() {
    int n, e;
    int adj[MAX][MAX];

    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;

    inputEdgesToAdjMatrix(adj, n, e);

    int start;
    cout << "Enter starting node: ";
    cin >> start;

    for (int i = 0; i < n; i++) visited[i] = 0;
    time_counter = 0;

    cout << "\nDFS Traversal: ";
    DFS(adj, n, start - 1);
    // Handle disconnected components
    for (int i = 0; i < n; i++) {
        if (visited[i] != 1) {
            cout << "\n(disconnected, new DFS from " << i + 1 << ") ";
            DFS(adj, n, i);
        }
    }

    cout << "\n\nNode\tStart\tFinish\n";
    for (int i = 0; i < n; i++)
        cout << i+1 << "\t" << startTime[i] << "\t" << finishTime[i] << "\n";

    return 0;
}

```

WHEN TO USE DFS (Scenario Triggers)

- ▶ Question mentions: exploring "as deep as possible" before backtracking
- ▶ Detecting a cycle in a graph (directed or undirected)
- ▶ Checking connectivity / finding connected components
- ▶ Topological sorting (DFS-based version)
- ▶ Finding articulation points, bridges, or strongly connected components
- ▶ Maze/pathfinding where you need ANY path, not the shortest one
- ▶ Question says "go deep first", "backtrack", or gives a stack-based hint

DFS vs BFS — Which One?

- ▶ Need SHORTEST path (unweighted graph)? → use BFS instead
- ▶ Need to explore full depth / detect cycles / topological order? → DFS
- ▶ Graph is very deep and narrow? → DFS uses less memory (stack vs queue)
- ▶ Graph is wide and shallow, dense? → notes say prefer BFS for dense graphs

MANUAL TRACE METHOD (How to Solve by Hand)

Use this exact process if the exam gives you a graph and asks you to trace DFS:

- ▶ 1. Pick the start node. Mark it visited. Write it in the Output list.
- ▶ 2. Look at the start node's neighbours in order (usually alphabetical / smallest index first).
- ▶ 3. Go to the FIRST unvisited neighbour. Mark visited, add to Output, push to stack (mentally).
- ▶ 4. Repeat step 2-3 from this new node — always go DEEPER first.
- ▶ 5. If a node has NO unvisited neighbours left, backtrack (pop) to the previous node and try its next neighbour.
- ▶ 6. Stop when the stack is empty. If nodes remain unvisited, graph is disconnected — restart DFS from any unvisited node.

Track Start/Finish time: every time you VISIT a node, write down a running counter (Start). Every time you POP/backtrack from a node (no more neighbours), write the counter again (Finish).

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Disconnected graphs — if not all nodes get visited from one start, you MUST restart DFS from an unvisited node. Don't just stop.

Directed vs Undirected — code only sets $\text{adj}[u][v]=1$ by default (directed). For undirected graphs, you must also set $\text{adj}[v][u]=1$.

1-indexed input, 0-indexed storage — teacher's code takes node 1,2,3... from user but stores as 0,1,2... internally. Easy to make an off-by-one mistake.

Edge classification (Tree/Forward/Back/Cross) — Back edge = pointing to an ancestor still on the stack (means a CYCLE in directed graphs). This is a favorite scenario-question trap.

Start/Finish time table — a common question format: "give the S/F time for each node" — practice this by hand, not just running code mentally.

EXAMPLE 1 — Small Connected Graph (Trace by Hand)

Graph (undirected): A-B, A-C, B-D, B-E, C-F, E-F

Start node: A

Step-by-step trace:

- ▶ Visit A (Start=1) → neighbours: B, C
- ▶ Visit B (Start=2) → neighbours: A(visited), D, E
- ▶ Visit D (Start=3) → no unvisited neighbours → Finish D=4
- ▶ Back to B → visit E (Start=5) → neighbours: B(visited), F
- ▶ Visit F (Start=6) → neighbours: C, E(visited)
- ▶ Visit C (Start=7) → neighbours: A(visited), F(visited) → Finish C=8
- ▶ Back to F → no more → Finish F=9
- ▶ Back to E → no more → Finish E=10
- ▶ Back to B → no more → Finish B=11
- ▶ Back to A → no more → Finish A=12

Output order: A, B, D, E, F, C

EXAMPLE 2 — Disconnected Graph (Restart Scenario)

Graph: A-B, C-D (two separate components)

Start node: A → DFS visits only A, B. Nodes C, D remain unvisited.

Exam expects you to say: "Since C is unvisited after the first DFS call, restart DFS from C." Final Output: A, B, C, D (two separate DFS calls combined).

EXAMPLE 3 — Cycle Detection Scenario (CP-style twist)

Scenario: "A network admin wants to know if adding a new cable between Router X and Router Y creates a loop in the network."

How to apply DFS here: run DFS from any node. If you reach a node that is already visited AND is currently an ancestor on the current path (still on stack) — that's a cycle. This maps directly to the "Back Edge" concept from class notes.

EXAMPLE 4 — Directed Graph Twist (Medium/Hard style)

Graph (directed): A→F, F→G, G→H, and also A→F, F→G directly again shown from notes (two paths reaching H)

Key exam question pattern: "Is there an edge classification issue here?" — if a path revisits a node already fully finished (Finish time set) via a different route, that's a FORWARD or CROSS edge, not a cycle. Only back-to-an-ancestor-on-stack counts as a cycle (Back Edge).

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER**Before you finalize any DFS answer, check:**

- Did I mark start/finish times correctly (only marks when visiting / when fully backtracking)?
- Did I handle disconnected components with a restart?
- Directed graph — did I check for back edges (cycle) correctly, not just "already visited"?
- Did I follow the correct neighbour order (usually smallest index / alphabetical first)?